JV32 RV32IMAC System-on-Chip Datasheet

JV32 Development Team

Version 1.1, 2026-04-08

Table of Contents

1. Introduction . . . 2
   1.1. Overview . . . 2
   1.2. Key Features . . . 2
   1.3. Document Organisation . . . 3
2. System Architecture . . . 4
   2.1. Block Diagram . . . 4
   2.2. System Components . . . 4
      2.2.1. Processor Core (`jv32_core`) . . . 4
      2.2.2. TCM: IRAM and DRAM (`jv32_top`) . . . 5
      2.2.3. Merged AXI4-Lite Master . . . 5
      2.2.4. AXI4-Lite Slave . . . 5
      2.2.5. Peripheral AXI Crossbar . . . 6
3. Memory Map . . . 7
   3.1. Address Space Overview . . . 7
   3.2. Address Decode Rules . . . 7
      3.2.1. TCM Decode (inside `jv32_top`) . . . 7
      3.2.2. Peripheral Crossbar Decode . . . 8
4. Processor Core Specification . . . 9
   4.1. RV32IMAC ISA Support . . . 9
   4.2. Pipeline Architecture . . . 9
   4.3. Forwarding and Hazards . . . 10
   4.4. Branch Prediction . . . 10
      4.4.1. BTFNT Algorithm . . . 10
      4.4.2. Performance Impact . . . 11
      4.4.3. BP_EN=0 Fallback . . . 11
   4.5. Performance Characteristics . . . 11
   4.6. CSR (Control and Status Registers) . . . 12
   4.7. Exception and Interrupt Handling . . . 14
      4.7.1. Exception Types . . . 14
      4.7.2. Interrupt Types . . . 14
      4.7.3. Trap Handling Sequence . . . 15
5. TCM Specification . . . 16
   5.1. Overview . . . 16
   5.2. Access Priorities . . . 16
   5.3. TCM Timing . . . 17

5.4. ELF Loading (Pre-Simulation) . . . 17
6. Peripheral Specifications . . . 18
6.1. UART (Universal Asynchronous Receiver/Transmitter) . . . 18
6.1.1. Overview . . . 18
6.1.2. Register Map . . . 18
6.1.3. Baud Rate Configuration . . . 19
6.1.4. Interrupt Behaviour . . . 20
6.1.5. External Pins . . . 20
6.2. CLIC (Core-Local Interrupt Controller) . . . 20
6.2.1. Overview . . . 20
6.2.2. Register Map . . . 21
6.2.3. Timer Operation . . . 21
6.2.4. Software Interrupt Operation . . . 22
6.2.5. External Interrupt Operation . . . 22
6.3. Magic Device (Simulation Only) . . . 22
6.3.1. Overview . . . 22
6.3.2. Register Map . . . 23
6.3.3. AXI Response Behaviour . . . 23
6.3.4. Usage Example . . . 23
7. AXI Interconnect and Bus Protocol . . . 24
7.1. AXI4-Lite Overview . . . 24
7.2. `jv32_top` AXI Master Port (`m_axi_*`) . . . 24
7.3. `jv32_top` AXI Slave Port (`s_axi_*`) . . . 25
7.4. Crossbar Architecture . . . 25
7.5. Bus State Machines (inside `jv32_top`) . . . 25
8. Electrical and Timing Characteristics . . . 27
8.1. Clock and Reset . . . 27
8.2. Timing Parameters . . . 27
8.3. Top-Level Parameters . . . 27
9. Programming Guide . . . 29
9.1. Boot Sequence . . . 29
9.2. Stack Configuration . . . 29
9.3. Interrupt Handling . . . 29
9.3.1. Setting Up Interrupts . . . 29
9.3.2. Trap Handler . . . 30
9.4. UART Usage . . . 31
9.5. Simulation Debugging . . . 31

9.5.1. Magic Device Console . . . 31
9.5.2. Waveform Capture . . . 31
9.5.3. Instruction Trace . . . 32
9.6. Performance Optimisation Tips . . . 32
10. Conclusion . . . 33
Appendix A: Register Quick Reference . . . 34
A.1. CLIC Registers . . . 34
A.2. UART Registers . . . 34
A.3. Magic Device Registers . . . 34
Appendix B: Interrupt Vector Table . . . 35
Appendix C: Pin Descriptions . . . 36
C.1. Clock and Reset . . . 36
C.2. UART Interface . . . 36
C.3. External Interrupt Interface . . . 36
C.4. TCM AXI Slave Interface . . . 36

This document provides comprehensive technical specifications for the JV32 System-on-Chip (SoC), a 32-bit RISC-V processor platform targeting embedded simulation and FPGA prototyping. The SoC integrates a 3-stage single-issue RV32IMAC pipeline core with 64 KB tightly-coupled instruction RAM (IRAM) and 64 KB tightly-coupled data RAM (DRAM), a merged AXI4-Lite master for out-of-TCM accesses, an AXI slave port for debug access to TCM, and three peripherals: UART (serial I/O), CLIC (interrupt controller with CLINT-compatible timer), and a simulation Magic device. The design is optimised for straightforward simulation and FPGA implementation.

Chapter 1. Introduction

1.1. Overview

The JV32 SoC is a complete minimal embedded processor system implementing the RISC-V RV32IMAC instruction set with Machine mode (M-mode) support. The system integrates:

- **RV32IMAC Processor Core** (`jv32_core`): Single-issue 3-stage pipeline (IF → EX → WB) with RVC expander, configurable fast multiply/divide/shift, branch predictor, and AMO state machine
- **Tightly Coupled Memories (TCM)**: 64 KB IRAM at `0x8000_0000` and 64 KB DRAM at `0xC000_0000`, instantiated inside `jv32_top` as byte-bank SRAM primitives with 1-cycle read latency
- **Merged AXI4-Lite Master**: Out-of-TCM accesses from both I-fetch and D-bus share one master port; D-bus has priority over I-fetch
- **AXI4-Lite Slave**: External agents (debug master, testbench DPI) can read/write TCM; core has absolute priority
- **Peripheral Subsystem** (3-slave AXI crossbar): UART, CLIC, Magic device
- **Interrupt System**: CLIC with CLINT-compatible 64-bit timer (`mtime`/`mtimecmp`), software interrupt (`msip`), and 16 independently-prioritised external interrupt lines

1.2. Key Features

Feature	Specification
ISA	RV32IMAC_Zicsr_Zifencei (`misa = 0x4000_1105`)
Pipeline	Single-issue 3-stage in-order (Fetch, Execute, Writeback); WB→EX same-cycle forwarding; 1-cycle load-use stall; BTFNT static branch predictor (`BP_EN=1`): 0-cycle penalty on correct prediction, 1-cycle penalty on misprediction
Instruction Memory	64 KB IRAM (TCM, 1-cycle access); out-of-TCM I-fetch via shared AXI4-Lite master
Data Memory	64 KB DRAM (TCM, 1-cycle access); IRAM is also D-accessible (IRAM address range); out-of-TCM via shared AXI4-Lite master
Privilege Modes	Machine (M-mode) only
Bus Protocol	AXI4-Lite (32-bit address, 32-bit data)

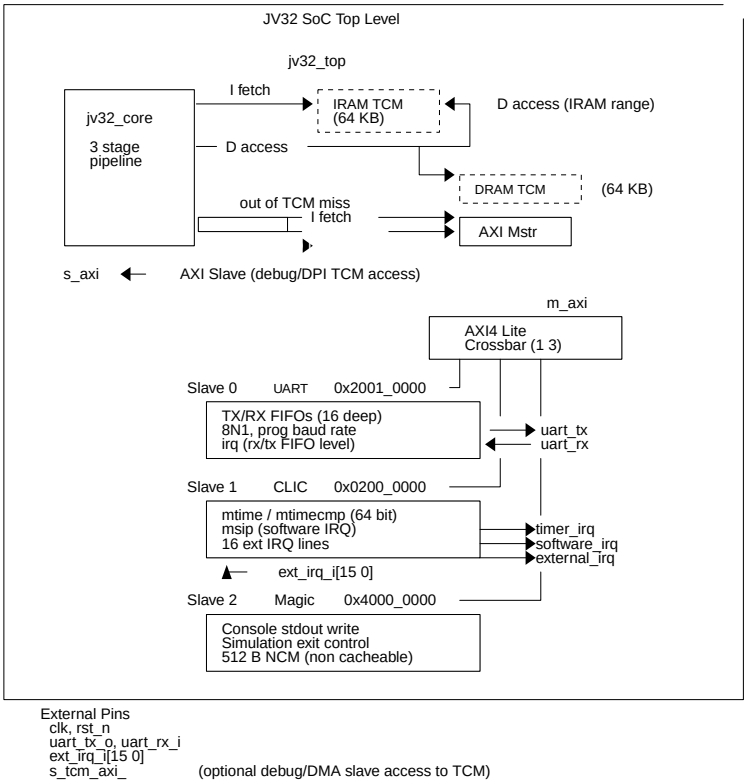
Feature	Specification
Clock Frequency	100 MHz (default, parameterisable)
Boot Address	`0x8000_0000` (IRAM base, parameterisable)
TCM Sizes	64 KB IRAM, 64 KB DRAM (each independently parameterisable, power-of-2)
Peripherals	UART (8N1, 16-deep FIFOs), CLIC/CLINT (64-bit timer, 16 external IRQs), Magic (simulation I/O)
Interrupts	Machine timer, machine software, 16 external CLIC lines (`ext_irq_i[15:0]`)
Multiply Latency	1 cycle (`FAST_MUL=1`), 1–32 cycles (`FAST_MUL=0`, CLZ-reduced on multiplier)
Divide Latency	1 cycle (`FAST_DIV=1`), 1–32 cycles (`FAST_DIV=0`, CLZ-reduced on dividend)
Implementation	RTL simulation (Verilator), FPGA synthesis supported

1.3. Document Organisation

- **Section 2**: System architecture and block diagram
- **Section 3**: Memory map and address decode
- **Section 4**: Processor core specification
- **Section 5**: TCM specification
- **Section 6**: Peripheral specifications (UART, CLIC, Magic)
- **Section 7**: AXI interconnect
- **Section 8**: Electrical and timing characteristics
- **Section 9**: Programming guide
- **Appendices**: Register quick reference, interrupt vectors, pin descriptions

Chapter 2. System Architecture

2.1. Block Diagram



2.2. System Components

2.2.1. Processor Core (`jv32_core`)

The RV32IMAC processor core implements a single-issue 3-stage in-order pipeline:

- **Instruction Fetch (IF)**: PC generation, RVC expansion, instruction fetch request
- **Execute (EX)**: Decode, register read, ALU, address calculation, CSR operations, hazard detection
- **Writeback (WB)**: Memory result, register writeback, branch redirect, exception handling

The core supports:

- Single-issue in-order execution; Slot-A all instructions; no dual-issue
- WB→EX same-cycle forwarding for RAW hazard resolution
- 1-cycle load-use stall (EX→WB stall + IF hold)

- 1-cycle branch/jump flush (EX squashes IF on misprediction or unconditional redirect)
- Multi-cycle stall for slow multiply/divide/barrel-shift when `FAST_MUL=0`/`FAST_DIV=0`/`FAST_SHIFT=0`
- AMO (LR/SC and AMO operations) via a 3-state machine (IDLE → LOAD_WAIT → STORE_WAIT)
- Configurable branch predictor (`BP_EN`, static not-taken fallback when disabled)
- CLIC-aware CSR unit with CLIC interrupt-level threshold (`mintthresh`), current level (`mintstatus`), vectored-trap base (`mtvt`), and next-interrupt fast-claim (`mnxti`)

2.2.2. TCM: IRAM and DRAM (`jv32_top`)

Both TCM regions are instantiated inside `jv32_top` as banks of `sram_1rw` byte-wide SRAM primitives:

- **IRAM**: 4 byte-banks (`gen_iram_byte[0..3].u_sram`), base `BOOT_ADDR` (default `0x8000_0000`), size `IRAM_SIZE` (default 64 KB)
- **DRAM**: 4 byte-banks (`gen_dram_byte[0..3].u_sram`), base `DRAM_BASE` (default `0xC000_0000`), size `DRAM_SIZE` (default 64 KB)
- **1-cycle read latency**: Address presented at cycle N returns data at cycle N+1
- **Contention resolution**: When I-fetch and D-access both target IRAM, D-access wins; I-fetch stalls 1 cycle
- **SRAM arbitration priority**: Core-D > Core-I > AXI Slave

2.2.3. Merged AXI4-Lite Master

Out-of-TCM accesses from I-fetch and D-bus share one AXI4-Lite master (`m_axi_*`):

- D-bus has priority over I-bus when both miss TCM simultaneously
- Both are served sequentially (not simultaneously)
- I-fetch AXI transaction is cancelled/replaced if a branch redirect arrives during the outstanding fetch

2.2.4. AXI4-Lite Slave

External agents (debug master, pre-simulation ELF loader via DPI) access TCM through the slave port (`s_axi_*`):

- Core has absolute priority; slave stalls until the target SRAM is idle for the current cycle

- The slave read-data register latches SRAM output on the first granted cycle and holds it until the bus handshake completes
- For pre-simulation ELF loading, the recommended method is DPI `mem_write_byte` (direct SRAM array access), which is faster than AXI slave transactions

2.2.5. Peripheral AXI Crossbar

A 1-to-3 AXI4-Lite crossbar routes out-of-TCM transactions from `jv32_top's merged master to three peripheral slaves. Address decode uses base/mask comparison (see Memory Map).

Chapter 3. Memory Map

3.1. Address Space Overview

The JV32 SoC implements a 32-bit address space (4 GB) with the following allocation:

Table 1. Complete Memory Map

Address Range	Device	Size	Description
`0x0200_0000` – `0x021F_FFFF`	CLIC	2 MB	RISC-V CLIC / CLINT-compatible interrupt controller; only first 64 KB used
`0x2001_0000` – `0x2001_FFFF`	UART	64 KB	Serial communication peripheral (8N1, 16-deep FIFOs)
`0x4000_0000` – `0x4FFF_FFFF`	Magic Device	256 MB (aperture)	Simulation console and exit control; only 0x2008 bytes used
`0x8000_0000` – `0x8000_FFFF`	IRAM (TCM)	64 KB	Tightly-coupled instruction + data RAM (inside `jv32_top`)
`0xC000_0000` – `0xC000_FFFF`	DRAM (TCM)	64 KB	Tightly-coupled data RAM (inside `jv32_top`)

TCM regions (IRAM, DRAM) are not accessible via the peripheral crossbar. They are decoded inside `jv32_top` before the AXI master port. Only accesses that miss TCM are forwarded to the crossbar.

3.2. Address Decode Rules

3.2.1. TCM Decode (inside `jv32_top`)

TCM is decoded combinatorially using power-of-2 masking:

Table 2. TCM Address Decode Functions

Region	Condition	Default
IRAM	`(addr & ~(IRAM_SIZE-1)) == BOOT_ADDR`	`BOOT_ADDR=0x8000_0000`, `IRAM_SIZE=65536`
DRAM	`(addr & ~(DRAM_SIZE-1)) == DRAM_BASE`	`DRAM_BASE=0xC000_0000`, `DRAM_SIZE=65536`

Accesses that do not match either TCM region are forwarded to the merged AXI master and routed to the peripheral crossbar.

3.2.2. Peripheral Crossbar Decode

The AXI crossbar (`axi_xbar`) decodes using base/mask:

Table 3. Peripheral Crossbar Address Decode

Slave	Device	Base	Mask	Notes
0	UART	`0x2001_0000`	`0xFFFF_FF00`	256-byte aperture; only 7 registers used
1	CLIC	`0x0200_0000`	`0xFFE0_0000`	2 MB aperture; CLINT registers + CLIC interrupt array
2	Magic	`0x4000_0000`	`0xF000_0000`	256 MB aperture; only 0x1200 bytes used

Unmapped addresses return AXI `DECERR` on reads and `DECERR` on writes.

Chapter 4. Processor Core Specification

4.1. RV32IMAC ISA Support

Table 4. Supported ISA Extensions

Extension	Description
RV32I	Base integer instruction set (32-bit)
M	Integer multiply and divide, including MULH, MULHU, MULHSU, DIV, DIVU, REM, REMU
A	Atomic instructions: LR.W/SC.W, AMO{SWAP,ADD,XOR,AND,OR,MIN,MAX,MINU,MAXU}.W
C (Zca)	16-bit compressed instructions; expanded to 32-bit by `jv32_rvc` before the Execute stage
Zicsr	CSR access instructions: CSRRW, CSRRS, CSRRC, CSRRWI, CSRRSI, CSRRCI
Zifencei	Instruction-fetch fence (`FENCE.I`): flushes the IF stage and re-fetches from PC+4

Machine mode (M-mode) only. User mode and Supervisor mode are not implemented.

4.2. Pipeline Architecture

The core uses a single-issue 3-stage in-order pipeline:

Table 5. Pipeline Stages

Stage	Name	Function
IF	Instruction Fetch	Generate PC; present address to TCM or AXI master; pass fetched instruction through `jv32_rvc` RVC expander
EX	Execute	Decode opcode; read register file; execute ALU/branch/load-address/store-address/CSR; detect hazards; compute branch target; AMO state machine
WB	Writeback	Receive memory response; write result to register file; handle exceptions; redirect pipeline on branch/jump/exception/MRET/interrupt

4.3. Forwarding and Hazards

Table 6. Pipeline Hazard Handling

Hazard	Type	Description
RAW (register read-after-write)	Forwarding	WB result forwarded to EX operand inputs in the same cycle. No stall required for ALU→ALU dependency.
Load-use	Stall (1 cycle)	Load in EX presents address; data arrives in WB. The following EX-stage instruction must stall 1 cycle waiting for WB to complete. IF is also held.
Branch / Jump misprediction	Flush (1 cycle)	When EX detects a taken branch or unconditional jump, the IF instruction in flight is squashed (1 pipeline bubble).
Multi-cycle ALU (mul/div/shift)	Stall (multiple cycles)	When `FAST_MUL=0`, `FAST_DIV=0`, or `FAST_SHIFT=0`, the entire pipeline stalls until the operation completes. Serial multiply and divide use CLZ to reduce iteration count (1–32 cycles depending on operand magnitude); serial shift iterates 0–31 cycles depending on shift amount.
AMO (LR/SC, AMO operations)	Stall	Two-phase read-modify-write sequence: EX issues AXI read, waits for response (LOAD_WAIT), then issues AXI write, waits for B-channel (STORE_WAIT). Pipeline stalls throughout.
FENCE / FENCE.I	Stall / Flush	`FENCE.I` flushes the IF stage and resumes from the sequential PC; `FENCE` is a no-op in the RV32IMAC single-issue in-order pipeline (all previous stores are observable before any subsequent load).

4.4. Branch Prediction

When `BP_EN=1` (default), the core implements a **Backward-Taken / Forward-Not-Taken (BTFNT)** static branch predictor. When `BP_EN=0`, all branches are predicted not-taken and resolved in EX.

4.4.1. BTFNT Algorithm

The predictor operates at the output of the RVC expander (end of IF stage) before the IF→EX latch is updated:

1. **Opcode detection**: The 32-bit expanded instruction is inspected for `opcode == 7'b110_0011` (RISC-V BRANCH group: `BEQ`, `BNE`, `BLT`, `BGE`, `BLTU`, `BGEU`).
2. **Direction heuristic**: The B-format immediate sign bit (`instr[31]`) is tested. A set sign bit means a negative offset (branch target is behind the current PC) → **predict taken**. A clear

sign bit means a positive offset (branch target is ahead) → **predict not-taken**.

3. **Redirect**: When a backward branch is predicted taken, `pc_if` is steered to the computed branch target *before* the instruction is latched into EX, and the RVC buffer is flushed to discard the speculatively fetched sequential instruction. The `bp_taken` field in the IF→EX pipeline register records whether the prediction was taken.
4. **EX correction**: When EX resolves the branch, one of three outcomes applies:
   - **Correct taken** (`bp_taken && branch_taken`): no redirect, 0-cycle penalty.
   - **Correct not-taken** (`!bp_taken && !branch_taken`): no redirect, 0-cycle penalty.
   - **Mispredicted taken** (`bp_taken && !branch_taken`): EX redirects `pc_if` to `pc_ex + instr_size` (2 for compressed, 4 for 32-bit), squashing the wrongly-fetched target instruction. 1-cycle penalty.
   - **Mispredicted not-taken** (`!bp_taken && branch_taken`): EX redirects `pc_if` to the branch target, squashing the sequential fetch. 1-cycle penalty.

4.4.2. Performance Impact

Backward branches dominate loop control flow, so BTFNT improves average CPI for loop-heavy code compared to a simple predict-not-taken strategy. The worst case is identical to predict-not-taken (1-cycle penalty).

4.4.3. BP_EN=0 Fallback

When `BP_EN=0`, the predictor logic is gated: `bp_redirect` is always 0, all branches are treated as not-taken in IF, and EX redirects the PC whenever a branch is actually taken (1-cycle penalty for every taken branch).

4.5. Performance Characteristics

Table 7. Performance Metrics

Metric	Typical Value
CPI (Cycles Per Instruction)	~1.0–1.1 (TCM hit, no branch; depends on instruction mix)
TCM Read Latency (I-fetch or D-load)	1 cycle (registered SRAM output)
Load-use Penalty	1 cycle stall

Metric	Typical Value
Branch Flush Penalty	0 cycles (correct BTFNT prediction) / 1 cycle (misprediction or `BP_EN=0` taken branch)
Multiply Latency	1 cycle (`FAST_MUL=1`), 1–32 cycles (`FAST_MUL=0`, CLZ-reduced on multiplier)
Divide Latency	1 cycle (`FAST_DIV=1`), 1–32 cycles (`FAST_DIV=0`, CLZ-reduced on dividend)
Barrel Shift Latency	1 cycle (`FAST_SHIFT=1`), up to 31 cycles (`FAST_SHIFT=0`)
AXI Out-of-TCM Read Latency	3–10+ cycles (depends on peripheral/AXI round-trip)
IRAM contention (D vs I access)	1 extra cycle stall on I-fetch (D-path wins)

4.6. CSR (Control and Status Registers)

The core implements Machine-mode CSRs per RISC-V Privileged Specification v1.11, plus CLIC extensions:

Table 8. Implemented CSRs

Address	Name	Access	Description
0x300	mstatus	R/W	Machine status: MIE[3], MPIE[7], MPP[12:11]=11 (M-mode always)
0x301	misa	R/O	ISA and extensions: `0x4000_1105` (RV32IMAC: A[0], C[2], I[8], M[12])
0x304	mie	R/W	Machine interrupt-enable: MSIE[3], MTIE[7], MEIE[11]
0x305	mtvec	R/W	Machine trap-handler base address (bits[1:0] = mode: 00=Direct, 01=Vectored)
0x307	mtvt	R/W	CLIC vector table base address (bit[1:0] must be 0; 64-byte aligned)
0x340	mscratch	R/W	Machine scratch register (software use)
0x341	mepc	R/W	Machine exception program counter (PC of interrupted/excepted instruction)
0x342	mcause	R/W	Machine trap cause: bit[31]=interrupt flag, bits[30:0]=cause code

Address	Name	Access	Description
0x343	mtval	R/W	Machine trap value (faulting address for memory exceptions, 0 otherwise)
0x344	mip	R/W	Machine interrupt-pending: MSIP[3], MTIP[7], MEIP[11]
0x345	mnxti	R/W	CLIC next-interrupt: reads next pending CLIC interrupt (above mintthresh) and claims it atomically; returns 0 if none pending
0x347	mintthresh	R/W	CLIC interrupt threshold (8-bit): only CLIC interrupts with level > mintthresh are taken
0xB00	mcycle	R/W	Machine cycle counter (lower 32 bits)
0xB02	minstret	R/W	Machine instructions-retired counter (lower 32 bits)
0xB80	mcycleh	R/W	Machine cycle counter (upper 32 bits)
0xB82	minstreth	R/W	Machine instructions-retired counter (upper 32 bits)
0xC00	cycle	R/O	Unprivileged alias of mcycle (lower 32 bits)
0xC01	time	R/O	Unprivileged alias (lower 32 bits; maps to CLIC mtime)
0xC02	instret	R/O	Unprivileged alias of minstret (lower 32 bits)
0xF11	mvendorid	R/O	Vendor ID (returns `0x0000_0000`)
0xF12	marchid	R/O	Architecture ID (returns `0x0000_0000`)
0xF13	mimpid	R/O	Implementation ID (returns `0x0000_0001`)
0xF14	mhartid	R/O	Hardware thread ID (returns `0x0000_0000`)
0xFB1	mintstatus	R/O	CLIC interrupt status: bits[31:24] = mil (machine interrupt level currently servicing)

4.7. Exception and Interrupt Handling

4.7.1. Exception Types

Table 9. Exception Causes (mcause[31]=0)

Code	Name	Description
0	Instruction address misaligned	PC not 4-byte aligned (2-byte alignment supported for RVC; 4-byte required for non-C instructions)
1	Instruction access fault	I-fetch AXI error response (SLVERR or DECERR from the AXI master)
2	Illegal instruction	Undefined or unsupported opcode
3	Breakpoint	EBREAK instruction executed
4	Load address misaligned	Load address not naturally aligned to the access size
5	Load access fault	Load AXI error response
6	Store/AMO address misaligned	Store or AMO address not naturally aligned
7	Store/AMO access fault	Store or AMO AXI error response
8	Environment call from U-mode	ECALL executed in U-mode (not normally reached; M-mode only system)
11	Environment call from M-mode	ECALL executed in M-mode

4.7.2. Interrupt Types

Table 10. Interrupt Causes (mcause[31]=1)

Code	Name	Source
3	Machine software interrupt	CLIC MSIP register (`msip[0]`)
7	Machine timer interrupt	CLIC: `mtime >= mtimecmp`
11	Machine external interrupt	CLIC: any enabled pending external IRQ (`ext_irq_i[15:0]`) with level > `mintthresh`

4.7.3. Trap Handling Sequence

On exception or interrupt:

1. `mstatus.MPIE ← mstatus.MIE`; `mstatus.MIE ← 0` (disable interrupts)
2. `mstatus.MPP ← 2'b11` (M-mode)
3. `mepc ← PC of interrupted/excepted instruction`
4. `mcause ← {1'b1/1'b0, cause_code}`
5. `mtval ← faulting_address` (or 0)
6. PC redirected to `mtvec` (direct mode) or `mtvec + 4*cause` (vectored mode)

MRET restores: `mstatus.MIE ← mstatus.MPIE`; `mstatus.MPIE ← 1`; PC ← `mepc`.

With CLIC, vectored interrupts redirect via `mtvt[base] + 4*clic_cause` and update `mintstatus.mil` with the claimed interrupt level.

Chapter 5. TCM Specification

5.1. Overview

Both IRAM and DRAM are physically located inside `jv32_top` as arrays of four byte-wide `sram_1rw` primitive instances:

Table 11. TCM Configuration

Parameter	Default Value
IRAM base address	`BOOT_ADDR = 32'h8000_0000`
IRAM size	`IRAM_SIZE = 65536` bytes (64 KB); power-of-2
DRAM base address	`DRAM_BASE = 32'hC000_0000`
DRAM size	`DRAM_SIZE = 65536` bytes (64 KB); power-of-2
Read latency	1 cycle (registered output: addr at cycle N → data at cycle N+1)
Write behaviour	Written at rising edge of cycle N; pipeline sees effect from cycle N onwards (NO_CHANGE read mode: rdata not updated on write cycles)
Data width	32 bits (4 byte banks, each 8-bit wide, byte-enable write)
DPI hierarchy (IRAM)	`u_soc.u_jv32.gen_iram_byte[b].u_sram.mem[word_index]` (b = 0..3)
DPI hierarchy (DRAM)	`u_soc.u_jv32.gen_dram_byte[b].u_sram.mem[word_index]` (b = 0..3)

5.2. Access Priorities

When multiple requestors target the same SRAM in the same cycle:

Table 12. SRAM Arbitration Priority

Priority	Requestor	Notes
1 (highest)	Core D-path	D-access (load, store, AMO) always wins
2	Core I-path (I-fetch)	If D-path does not need the SRAM this cycle
3 (lowest)	AXI Slave	Only granted when neither I-path nor D-path accesses the target SRAM

5.3. TCM Timing

Table 13. Timing Summary

Transaction	Behaviour
I-fetch (IRAM hit)	Addr presented at cycle N; instruction available at cycle N+1 (1-cycle latency)
D-read (IRAM or DRAM hit)	Addr presented at cycle N; data available at cycle N+1; pipeline inserts 1-cycle load stall
D-write (IRAM or DRAM hit)	Data written at rising edge of cycle N; write acknowledged at cycle N+1 (1 AXI cycle)
IRAM contention (D + I both need IRAM)	D-path wins; I-fetch stalls 1 extra cycle
Stale I-fetch after redirect	`imem_flush` signal (= `rvc_flush`) gates the TCM response; cycle N+1 response is suppressed when a branch/exception redirect occurred at cycle N

5.4. ELF Loading (Pre-Simulation)

For Verilator simulation, program images are loaded directly into TCM SRAM arrays via DPI-C before simulation starts. The testbench `tb_jv32_soc.cpp` calls `load_program()` from `elfloader.cpp`:

```
// Load ELF into TCM via DPI mem_write_byte
void mem_write_byte(uint32_t addr, uint8_t data) {
    if (addr >= IRAM_BASE && addr < IRAM_BASE + IRAM_SIZE) {
        uint32_t offset = addr - IRAM_BASE;
        uint32_t word   = offset >> 2;
        uint32_t bank   = offset & 3;
        // Writes: u_soc.u_jv32.gen_iram_byte[bank].u_sram.mem[word]
    } else if (addr >= DRAM_BASE && addr < DRAM_BASE + DRAM_SIZE) {
        uint32_t offset = addr - DRAM_BASE;
        uint32_t word   = offset >> 2;
        uint32_t bank   = offset & 3;
        // Writes: u_soc.u_jv32.gen_dram_byte[bank].u_sram.mem[word]
    }
}
```

Chapter 6. Peripheral Specifications

6.1. UART (Universal Asynchronous Receiver/Transmitter)

6.1.1. Overview

The UART peripheral provides serial communication with configurable baud rate and 8N1 format.

Table 14. UART Features

Feature	Specification
Base Address	`0x2001_0000`
Aperture	64 KB (only first 0x18 bytes used; accesses beyond 0x18 return SLVERR)
Data Format	8 bits, no parity, 1 stop bit (8N1)
Baud Rate	Configurable via `BAUD_RATE` parameter (default `115_200`); compile-time fixed (`CLKS_PER_BIT = CLK_FREQ / BAUD_RATE`)
TX FIFO Depth	16 entries (parameterisable: `FIFO_DEPTH`, must be power-of-2)
RX FIFO Depth	16 entries (same as TX)
Interrupt	Level-triggered; asserted when `(IE & IS) != 0`

6.1.2. Register Map

Table 15. UART Registers (base `0x2001_0000`)

Offset	Name	Access	Description
0x00	DATA	R/W	**Write**: push byte (bits[7:0]) to TX FIFO (silently dropped if FIFO full) **Read**: pop byte from RX FIFO (returns 0 if empty)
0x04	STATUS	R/O	**[0]** `tx_full` — TX FIFO full (cannot write) **[1]** `tx_full` (duplicate flag; same as bit[0]) **[2]** `rx_not_empty` — RX FIFO has data **[3]** `rx_full` — RX FIFO full (overrun possible)
0x08	IE (Interrupt Enable)	R/W	**[0]** `rx_not_empty_ie` — enable RX not-empty interrupt **[1]** `tx_empty_ie` — enable TX empty interrupt

Offset	Name	Access	Description
0x0C	IS (Interrupt Status)	R/O	**[0]** `rx_not_empty` — RX FIFO non-empty (level) **[1]** `tx_empty` — TX FIFO empty (level)
0x10	LEVEL	R/O	**[15:0]** `rx_count` — number of bytes in RX FIFO **[31:16]** `tx_count` — number of bytes in TX FIFO
0x14	CTRL	R/W	**[0]** `loopback_en` — internal TX→RX loopback (for testing)

Interrupt asserts when `(IE & IS) != 0`. There is no explicit interrupt-clear register; the interrupt de-asserts automatically when the FIFO condition clears (e.g. RX FIFO drained, TX FIFO refilled).

6.1.3. Baud Rate Configuration

The baud rate is determined at elaboration time:

```
CLKS_PER_BIT = CLK_FREQ / BAUD_RATE
```

Hardware constraint: `CLKS_PER_BIT >= 4` (absolute minimum for RX centre sampling).

Recommended: `CLKS_PER_BIT >= 8` for production use.

Table 16. Common Baud Rates at 100 MHz

Baud Rate	CLKS_PER_BIT	Notes
9600	10417	Very slow, large margin
115200	868	Standard; large margin
921600	109	Fast; good margin
1 Mbaud	100	Round number; good margin
12.5 Mbaud	8	Very fast; minimum recommended
25 Mbaud	4	Maximum (hardware minimum; lab use only)

Baud rates requiring `CLKS_PER_BIT < 4` will not be received correctly.

6.1.4. Interrupt Behaviour

- **RX interrupt** (`IS[0]`): Fires whenever RX FIFO is non-empty. Stays asserted until all bytes are consumed.
- **TX interrupt** (`IS[1]`): Fires whenever TX FIFO is empty. Stays asserted until software writes more data.

6.1.5. External Pins

Table 17. UART Pins

Signal	Direction	Description
`uart_tx_o`	Output	UART transmit data
`uart_rx_i`	Input	UART receive data (asynchronous to system clock)

6.2. CLIC (Core-Local Interrupt Controller)

6.2.1. Overview

The CLIC integrates the RISC-V CLINT (Core-Local Interruptor) timer and software interrupt registers with a 16-line external interrupt controller supporting per-interrupt enable, level, and priority.

Table 18. CLIC Features

Feature	Specification
Base Address	`0x0200_0000`
Aperture	2 MB (CLIC spec aperture; only CLINT registers + first 64 CLICINT entries used)
CLINT Compatibility	Full `mtime`/`mtimecmp`/`msip` register layout at standard RISC-V CLINT offsets
Timer Counter	64-bit free-running `mtime`, increments every clock cycle
Timer Compare	64-bit `mtimecmp`; timer IRQ fires when `mtime >= mtimecmp`
Software Interrupt	`msip[0]`: write 1 to assert, write 0 to clear
External Interrupt Lines	16 (`NUM_IRQ=16`), routed from `ext_irq_i[15:0]`, each with individual enable/level/priority via `CLICINT[n]`

Feature	Specification
Interrupt Outputs	`timer_irq_o`, `software_irq_o`, `clic_irq_o` (aggregated external), `clic_level_o[7:0]`, `clic_prio_o[7:0]`, `clic_vector_pc_o[31:0]`

6.2.2. Register Map

Table 19. CLIC / CLINT Registers (base `0x0200_0000`)

Offset	Name	Access	Description
0x0000	MSIP	R/W	Machine Software Interrupt Pending. `[0]=msip`: write 1 to assert software interrupt; write 0 to clear.
0x4000	MTIME_LO	R/W	`mtime[31:0]` — lower 32 bits of 64-bit free-running timer
0x4004	MTIME_HI	R/W	`mtime[63:32]` — upper 32 bits
0x4008	MTIMECMP_LO	R/W	`mtimecmp[31:0]` — lower 32 bits of timer compare
0x400C	MTIMECMP_HI	R/W	`mtimecmp[63:32]` — upper 32 bits. Timer IRQ fires when `mtime >= mtimecmp`.
0x1000 + n×4	CLICINT[n] (n=0..15)	R/W	Per-interrupt control word (32-bit): **[0]** `ip` (read-only) — interrupt pending; reflects `ext_irq_i[n]` **[1]** `ie` — interrupt enable (1=enabled) **[15:8]** `attr` — interrupt attributes (edge/level, polarity) **[23:16]** `ctl` — interrupt priority/level (8-bit; higher = more urgent)

MSIP, MTIME, and MTIMECMP offsets match the standard RISC-V CLINT specification for single-hart systems (compatible with `kv_clint_*` SDK functions).

6.2.3. Timer Operation

The 64-bit `mtime` counter increments by 1 every system clock cycle. The timer interrupt fires when `mtime >= mtimecmp` (unsigned 64-bit comparison). Software clears the timer interrupt by writing a new, larger value to `mtimecmp`.

```
// Set timer 1 ms from now (100 MHz clock)
uint64_t now = ((uint64_t)CLIC_MTIME_HI << 32) | CLIC_MTIME_LO;
```

```
uint64_t cmp = now + 100000;   // 1 ms at 100 MHz
CLIC_MTIMECMP_LO = (uint32_t)(cmp & 0xFFFFFFFF);
CLIC_MTIMECMP_HI = (uint32_t)(cmp >> 32);
```

Always write MTIMECMP_HI (set to `0xFFFFFFFF`) before writing MTIMECMP_LO when programming a new compare value, to avoid a spurious timer interrupt in the window between the two writes on a 64-bit counter.

6.2.4. Software Interrupt Operation

Write `MSIP[0] = 1` to assert machine software interrupt immediately, `0` to clear.

6.2.5. External Interrupt Operation

Each `ext_irq_i[n]` input is gateable via `CLICINT[n].ie`. When any enabled and pending external interrupt has `ctl` (priority level) greater than the current `mintthresh` CSR value, `clic_irq_o` is asserted to the core as an M-mode external interrupt.

The CLIC computes `clic_vector_pc_o` as `mtvt + 4 * irq_index` for vectored CLIC operation (requires `mtvec` in CLIC-vectored mode and `mtvt` programmed to a handler table).

6.3. Magic Device (Simulation Only)

6.3.1. Overview

The Magic Device provides simulation console I/O, simulation exit control, and a non-cacheable memory (NCM) window for bypass testing.

The Magic Device is for simulation only. It must not be synthesised for FPGA/ASIC production.

Table 20. Magic Device Features

Feature	Specification
Base Address	`0x4000_0000`
Console Write	Offset `0x0000`: write byte (bits[7:0]) to host `stdout`
Exit Control	Offset `0x0004`: write to terminate simulation. Value 1 = PASS (exit 0); value N ≠ 1 = FAIL (exit N via DPI `sim_request_exit`)

Feature	Specification
NCM Region	Offsets `0x1000`–`0x11FF`: 128 × 32-bit words (512 B) non-cacheable memory for uncached access testing

6.3.2. Register Map

Table 21. Magic Device Registers (base `0x4000_0000`)

Offset	Name	Access	Description
0x0000	CONSOLE	W/O	Write character (low byte) to simulation `stdout`. Reads return 0.
0x0004	EXIT	W/O	Write to terminate simulation. `1` = success (exit 0). Any other value N = failure (exit N). Uses DPI-C `sim_request_exit(N)`.
0x1000–0x11FF	NCM[0..127]	R/W	128 × 32-bit non-cacheable memory. Indexed by `addr[8:2]`. Byte strobes on writes apply to the full 32-bit word (no partial-word merge). Useful for testing non-cacheable I-fetch and D-access paths.

6.3.3. AXI Response Behaviour

`axi_magic` returns `OKAY` for decoded regions (`CONSOLE`, `EXIT`, NCM window). All other addresses within the Magic device AXI aperture return `SLVERR`.

6.3.4. Usage Example

```
#define MAGIC_CONSOLE  (*(volatile uint32_t *)0x40000000)
#define MAGIC_EXIT     (*(volatile uint32_t *)0x40000004)

void print_str(const char *s) {
    while (*s) MAGIC_CONSOLE = *s++;
}

void sim_pass(void) { MAGIC_EXIT = 1; }   // exit(0)
void sim_fail(int code) { MAGIC_EXIT = (code << 1) | 1; }
```

Chapter 7. AXI Interconnect and Bus Protocol

7.1. AXI4-Lite Overview

The JV32 SoC uses AXI4-Lite throughout (32-bit address, 32-bit data, no burst, no ID).

Table 22. AXI Protocol Summary

Feature	Specification
Protocol	AXI4-Lite (AMBA 4 subset: single-beat, no ARLEN/AWLEN/ARBURST, no IDs)
Data Width	32 bits
Address Width	32 bits
Write Strobes	4 bits (byte granularity)
Response Codes	`OKAY` (2'b00), `SLVERR` (2'b10), `DECERR` (2'b11)
Crossbar Topology	1 master (`jv32_top` merged output), 3 peripheral slaves

7.2. `jv32_top` AXI Master Port (`m_axi_*`)

The merged AXI master port carries all out-of-TCM traffic:

Table 23. AXI4-Lite Master Signals

Signal	Direction	Description
`m_axi_araddr[31:0]`	Output	Read address channel: address
`m_axi_arvalid`	Output	Read address valid
`m_axi_arready`	Input	Read address ready (slave accepts)
`m_axi_rdata[31:0]`	Input	Read data
`m_axi_rresp[1:0]`	Input	Read response (`OKAY`/`SLVERR`/`DECERR`)
`m_axi_rvalid`	Input	Read data valid
`m_axi_rready`	Output	Read data ready (master accepts)
`m_axi_awaddr[31:0]`	Output	Write address

Signal	Direction	Description
m_axi_awvalid	Output	Write address valid
m_axi_awready	Input	Write address ready
m_axi_wdata[31:0]	Output	Write data
m_axi_wstrb[3:0]	Output	Write byte strobes
m_axi_wvalid	Output	Write data valid
m_axi_wready	Input	Write data ready
m_axi_bresp[1:0]	Input	Write response
m_axi_bvalid	Input	Write response valid
m_axi_bready	Output	Write response ready

7.3. jv32_top AXI Slave Port (s_axi_*)

The TCM slave port has the same signal set as the master port (with directions flipped). All s_axi_* signals are exposed as SoC-level ports (s_tcm_* prefix in jv32_soc).

7.4. Crossbar Architecture

The axi_xbar module implements a 1-to-N address-decoder crossbar:

- **Address decode**: (addr & SLAVE_MASK[i]) == (SLAVE_BASE[i] & SLAVE_MASK[i]) for each slave in order; first match wins
- **Error response**: DECERR on both read and write for unmapped addresses
- **Peripheral protocol restriction**: All peripheral slaves are single-beat only; ARLEN/AWLEN are not forwarded; RLAST is hardcoded to 1'b1

7.5. Bus State Machines (inside jv32_top)

The merged AXI master is implemented as a state machine with the following states:

Table 24. AXI Master State Machine

State	Description
`BUS_IDLE`	Both I-fetch and D-access in TCM or idle. In IDLE, D-bus out-of-TCM check has priority over I-bus.
`DAR`	D-bus AXI read address phase: `m_axi_araddr` set to D-bus address; wait for `arready`.
`DR`	D-bus AXI read data phase: wait for `m_axi_rvalid`; return data to core D-path.
`DAW`	D-bus AXI write address+data phase: issue both AW and W channels (simultaneous when possible).
`DB`	D-bus AXI write response: wait for `m_axi_bvalid`.
`IAR`	I-bus AXI read address phase: `m_axi_araddr` set to I-fetch PC; wait for `arready`.
`IR`	I-bus AXI read data phase: wait for `m_axi_rvalid`; return instruction to core I-path.

Chapter 8. Electrical and Timing Characteristics

8.1. Clock and Reset

Table 25. Clock and Reset Specifications

Parameter	Value	Description
`clk`	Input	System clock (100 MHz nominal)
`CLK_FREQ`	`100_000_000`	Default clock frequency parameter (Hz); affects UART baud divisor
`rst_n`	Active-low asynchronous	Must be asserted (low) for ≥ 2 clock cycles before release
Reset recovery	2 cycles	Pipeline initialises within 2 cycles of `rst_n` deassertion

8.2. Timing Parameters

Table 26. Timing Characteristics (100 MHz, Tclk = 10 ns)

Parameter	Min	Max	Description
Setup time (AXI inputs)	2.0 ns	—	Input setup before clock edge
Hold time (AXI inputs)	1.0 ns	—	Input hold after clock edge
Clock-to-output (AXI outputs)	—	8.0 ns	Output valid after clock edge
TCM read latency	1 cycle	—	Registered SRAM output
Load-to-use latency	2 cycles	—	1 EX cycle + 1 stall cycle
Branch flush penalty	0 / 1 cycle	—	0 cycles on correct BTFNT prediction; 1 cycle on misprediction or when `BP_EN=0` and branch is taken
Interrupt entry latency	3 cycles	10 cycles	Depends on pipeline state and outstanding transactions

8.3. Top-Level Parameters

Table 27. `jv32_soc` Configurable Parameters

Parameter	Default	Description
`CLK_FREQ`	`100_000_000`	System clock frequency in Hz; used to compute UART baud divisor
`BAUD_RATE`	`115_200`	UART baud rate (must satisfy `CLK_FREQ / BAUD_RATE >= 4`)
`IRAM_SIZE`	`65536`	IRAM size in bytes (power-of-2; also sets TCM address decode mask)
`DRAM_SIZE`	`65536`	DRAM size in bytes (power-of-2)
`FAST_MUL`	`1`	`1`=single-cycle combinatorial multiply; `0`=33-cycle serial multiply
`FAST_DIV`	`1`	`1`=single-cycle combinatorial divide; `0`=33-cycle serial divide
`FAST_SHIFT`	`1`	`1`=single-cycle barrel shift; `0`=iterative shift (up to 31 cycles)
`BP_EN`	`1`	`1`=BTFNT static branch predictor enabled (backward branches predicted taken, forward branches predicted not-taken; 0-cycle penalty on correct prediction, 1-cycle on misprediction); `0`=all branches predicted not-taken (1-cycle penalty on every taken branch)
`BOOT_ADDR`	`32'h8000_0000`	Reset PC and IRAM base address
`DRAM_BASE`	`32'hC000_0000`	DRAM base address and TCM address decode start

Chapter 9. Programming Guide

9.1. Boot Sequence

The processor boots from `BOOT_ADDR` (default `0x8000_0000` = IRAM base). The reset vector is hardwired to `BOOT_ADDR & ~3` (word-aligned).

For simulation, the typical boot flow is:

1. ELF loaded into TCM via DPI (`mem_write_byte`) before simulation starts
2. Simulation starts, core fetches from `BOOT_ADDR` (mapped to IRAM, 1-cycle latency)
3. Startup code (`startup.S`) initialises stack pointer to top of DRAM
4. Hardware CSRs initialised: `mstatus`, `mtvec`, `mie`
5. Peripherals initialised (UART baud rate is compile-time; CLIC `mtimecmp` programmed)
6. `mstatus.MIE` set to enable interrupts
7. Jump to `main()`

9.2. Stack Configuration

Default linker script maps DRAM to data/BSS/stack:

```
Stack top: DRAM_BASE + DRAM_SIZE - 4    (e.g. 0xC000_FFFC)
Stack grows downward into DRAM
Data/BSS:  DRAM_BASE (loaded by startup.S from IRAM)
```

9.3. Interrupt Handling

9.3.1. Setting Up Interrupts

```c
#include "csr.h"

extern void trap_entry(void);  // defined in startup.S

void setup_interrupts(void) {
    // Set trap vector (direct mode: mtvec points to handler)
    write_csr(mtvec, (uint32_t)trap_entry);

    // Enable M-mode interrupts in mstatus
    set_csr(mstatus, 0x8);  // MIE bit

    // Enable timer + external + software interrupts in mie
    set_csr(mie, (1<<7) | (1<<11) | (1<<3));  // MTIE | MEIE | MSIE
```

```
    // Program CLIC mtimecmp for periodic 1ms timer (100 MHz)
    volatile uint32_t *mtime_lo    = (uint32_t *)0x02004000;
    volatile uint32_t *mtime_hi    = (uint32_t *)0x02004004;
    volatile uint32_t *mtimecmp_lo = (uint32_t *)0x02004008;
    volatile uint32_t *mtimecmp_hi = (uint32_t *)0x0200400C;

    // Set mtimecmp_hi first to avoid spurious IRQ
    *mtimecmp_hi = 0xFFFFFFFF;
    uint64_t now = ((uint64_t)*mtime_hi << 32) | *mtime_lo;
    uint64_t cmp = now + 100000;  // 1 ms @ 100 MHz
    *mtimecmp_lo = (uint32_t)cmp;
    *mtimecmp_hi = (uint32_t)(cmp >> 32);
}
```

9.3.2. Trap Handler

```
void trap_handler(uint32_t mcause_val) {
    if (mcause_val & 0x80000000) {
        // Interrupt
        uint32_t cause = mcause_val & 0x7FFFFFFF;
        switch (cause) {
        case 7:   // Machine timer interrupt
            handle_timer_irq();
            break;
        case 11:  // Machine external interrupt (CLIC)
            handle_clic_irq();
            break;
        case 3:   // Machine software interrupt
            handle_sw_irq();
            break;
        }
    } else {
        // Synchronous exception
        handle_exception(mcause_val);
    }
}

void handle_timer_irq(void) {
    // Clear by advancing mtimecmp
    volatile uint32_t *cmp_lo = (uint32_t *)0x02004008;
    volatile uint32_t *cmp_hi = (uint32_t *)0x0200400C;
    uint64_t cmp = ((uint64_t)*cmp_hi << 32) | *cmp_lo;
    cmp += 100000;  // next 1 ms
    *cmp_hi = 0xFFFFFFFF;  // prevent spurious IRQ during update
    *cmp_lo = (uint32_t)cmp;
    *cmp_hi = (uint32_t)(cmp >> 32);
}

void handle_clic_irq(void) {
    // CLIC vectored: read MNXTI to claim and get vector PC
    // Or: scan CLICINT[n].ip to find pending source
    for (int n = 0; n < 16; n++) {
        volatile uint32_t *clicint = (uint32_t *)(0x02001000 + n * 4);
        if (*clicint & 0x1) {
```

```
            // Source n pending — handle it
            // (IP bit auto-clears when ext_irq_i[n] deasserts)
        }
    }
}
```

9.4. UART Usage

```
#define UART_DATA   (*(volatile uint32_t *)0x20010000)
#define UART_STATUS (*(volatile uint32_t *)0x20010004)
#define UART_IE     (*(volatile uint32_t *)0x20010008)
#define UART_IS     (*(volatile uint32_t *)0x2001000C)

// Poll-mode transmit
void uart_putc(char c) {
    while (UART_STATUS & 0x1);  // wait until TX not full
    UART_DATA = c;
}

// Poll-mode receive
char uart_getc(void) {
    while (!(UART_STATUS & 0x4));  // wait until RX not empty
    return (char)(UART_DATA & 0xFF);
}

// Print string
void uart_puts(const char *s) {
    while (*s) uart_putc(*s++);
}
```

9.5. Simulation Debugging

9.5.1. Magic Device Console

```
#define MAGIC_CON (*(volatile uint32_t *)0x40000000)
#define MAGIC_EXIT (*(volatile uint32_t *)0x40000004)

void debug_putc(char c) { MAGIC_CON = c; }
void debug_puts(const char *s) { while (*s) debug_putc(*s++); }
void sim_pass(void) { MAGIC_EXIT = 1; }
void sim_fail(int code) { MAGIC_EXIT = code; }
```

9.5.2. Waveform Capture

Generate VCD waveform for GTKWave analysis:

```
make WAVE=1 rtl-hello
gtkwave build/tb_jv32_soc.vcd
```

Key signals to observe:

- **Pipeline**: `jv32_core.pc_if`, `jv32_core.pc_ex`, `jv32_core.pc_wb`
- **TCM**: `jv32_top.imem_req_valid`, `jv32_top.imem_resp_valid`, `jv32_top.dmem_req_valid`
- **AXI master**: `jv32_top.m_axi_arvalid`, `jv32_top.m_axi_rvalid`
- **Interrupts**: `jv32_soc.timer_irq`, `jv32_soc.clic_irq`

9.5.3. Instruction Trace

Software and RTL traces can be compared for regression:

```
make compare-hello
diff sim_trace.txt rtl_trace.txt
```

Trace format: `<cycle> <pc> <hex_instruction> <disassembly>`

9.6. Performance Optimisation Tips

1. **Minimise out-of-TCM accesses**: Place hot code in IRAM (`BOOT_ADDR` region) and data in DRAM (`DRAM_BASE` region) — both 1-cycle access
2. **Eliminate load-use stalls**: Reorder instructions so a load result is not used in the immediately following instruction
3. **Use fast ALU options**: `FAST_MUL=1`, `FAST_DIV=1`, `FAST_SHIFT=1` for single-cycle operations (area trade-off)
4. **Keep stack in DRAM**: All stack frames in DRAM ensure 1-cycle load/store latency
5. **Avoid IRAM D-access contention**: Keep data-heavy code paths away from IRAM address range to avoid I-fetch stalls from priority resolution

Chapter 10. Conclusion

This datasheet provides a complete reference for the JV32 RV32IMAC SoC, covering its 3-stage TCM-based pipeline architecture, memory map, peripheral specifications, AXI interconnect, and programming guide. For additional information refer to:

- **RISC-V Specifications**: https://riscv.org/technical/specifications/
- **AMBA AXI4-Lite Specification**: https://developer.arm.com/documentation/ihi0022/latest/
- **Project Repository Source**: `/home/kuoping/Projects/jv32/`
- **RTL Entry Point**: `rtl/jv32_soc.sv`, `rtl/jv32/jv32_top.sv`

Appendix A: Register Quick Reference

A.1. CLIC Registers

Address	Name	Access	Reset Value
0x0200_0000	MSIP	R/W	0x0000_0000
0x0200_4000	MTIME_LO	R/W	0x0000_0000
0x0200_4004	MTIME_HI	R/W	0x0000_0000
0x0200_4008	MTIMECMP_LO	R/W	0xFFFF_FFFF
0x0200_400C	MTIMECMP_HI	R/W	0xFFFF_FFFF
0x0200_1000 + n×4	CLICINT[n] (n=0..15)	R/W	0x0000_0000

A.2. UART Registers

Address	Name	Access	Reset Value
0x2001_0000	DATA	R/W	0x0000_0000
0x2001_0004	STATUS	R/O	0x0000_0002 (TX not full)
0x2001_0008	IE	R/W	0x0000_0000
0x2001_000C	IS	R/O	0x0000_0002 (TX empty)
0x2001_0010	LEVEL	R/O	0x0000_0000
0x2001_0014	CTRL	R/W	0x0000_0000

A.3. Magic Device Registers

Address	Name	Access	Reset Value
0x4000_0000	CONSOLE	W/O	—
0x4000_0004	EXIT	W/O	—
0x4000_1000–0x4000_11FC	NCM[0..127]	R/W	0x0000_0000

Appendix B: Interrupt Vector Table

Table 28. Machine-Mode Interrupt Vectors (mcause encoding)

Code	Type	Description	Source
0	Exception	Instruction address misaligned	Core
1	Exception	Instruction access fault	I-fetch AXI SLVERR/DECERR
2	Exception	Illegal instruction	Core decode
3	Exception	Breakpoint	EBREAK
4	Exception	Load address misaligned	Core
5	Exception	Load access fault	D-bus AXI SLVERR/DECERR
6	Exception	Store/AMO address misaligned	Core
7	Exception	Store/AMO access fault	D-bus AXI SLVERR/DECERR
8	Exception	ECALL from U-mode	(not reached in M-only system)
11	Exception	ECALL from M-mode	ECALL instruction
`0x8000_0003`	Interrupt	Machine software interrupt	CLIC MSIP[0]
`0x8000_0007`	Interrupt	Machine timer interrupt	CLIC `mtime >= mtimecmp`
`0x8000_000B`	Interrupt	Machine external interrupt	CLIC any `CLICINT[n].ie & ip` with level > mintthresh

`mcause[31]=1` indicates interrupt; `mcause[30:0]` is the cause code.

Appendix C: Pin Descriptions

C.1. Clock and Reset

Table 29. Clock and Reset Pins

Signal	I/O	Width	Description
`clk`	Input	1	System clock (100 MHz nominal)
`rst_n`	Input	1	Asynchronous active-low reset

C.2. UART Interface

Table 30. UART Pins

Signal	I/O	Width	Description
`uart_tx_o`	Output	1	UART transmit data
`uart_rx_i`	Input	1	UART receive data (asynchronous)

C.3. External Interrupt Interface

Table 31. External Interrupt Pins

Signal	I/O	Width	Description
`ext_irq_i[15:0]`	Input	16	External interrupt request lines, routed to CLIC `CLICINT[n].ip`; active-high, level-sensitive

C.4. TCM AXI Slave Interface

Table 32. TCM Slave Port Pins (exposed as `s_tcm_*` on `jv32_soc`)

Signal	I/O	Width	Description
`s_tcm_araddr[31:0]`	Input	32	Read address
`s_tcm_arvalid`	Input	1	Read address valid
`s_tcm_arready`	Output	1	Read address ready
`s_tcm_rdata[31:0]`	Output	32	Read data

Signal	I/O	Width	Description
`s_tcm_rresp[1:0]`	Output	2	Read response
`s_tcm_rvalid`	Output	1	Read data valid
`s_tcm_rready`	Input	1	Read data ready
`s_tcm_awaddr[31:0]`	Input	32	Write address
`s_tcm_awvalid`	Input	1	Write address valid
`s_tcm_awready`	Output	1	Write address ready
`s_tcm_wdata[31:0]`	Input	32	Write data
`s_tcm_wstrb[3:0]`	Input	4	Write byte strobes
`s_tcm_wvalid`	Input	1	Write data valid
`s_tcm_wready`	Output	1	Write data ready
`s_tcm_bresp[1:0]`	Output	2	Write response
`s_tcm_bvalid`	Output	1	Write response valid
`s_tcm_bready`	Input	1	Write response ready